

Chương 5

Thiết kế RTL kiến trúc Radix- 2^2 SDF FFT

Đồ án: Thiết kế bộ tăng tốc phần cứng cho FFT Radix-2/Radix-4

Ngày 26 tháng 5 năm 2026

Mục lục

5	Thiết kế RTL kiến trúc Radix-2 bình phương SDF FFT	2
5.1	Định hướng thiết kế và tham số hệ thống	2
5.2	Từ phương trình thuật toán sang khối RTL	3
5.3	Sơ đồ khối tổng thể FFT-256 R2 bình phương SDF	4
5.4	Nguyên lý tầng SDF cơ bản	5
5.5	Thiết kế BF2I – butterfly Radix-2 cơ bản	6
5.6	Thiết kế BF2II và bộ xoay tâm thường	6
5.7	Thiết kế bộ nhân phức CMUL	7
5.8	Twiddle ROM và sinh địa chỉ	8
5.9	Đường trễ và bộ nhớ phản hồi	9
5.10	Bộ điều khiển	10
5.11	Ước lượng tài nguyên	11
5.12	Tổng kết chương	12

Chương 5

Thiết kế RTL kiến trúc Radix-2² SDF FFT

Chương này trình bày cách chuyển thuật toán Radix-2² DIF FFT từ mô hình toán học sang kiến trúc RTL có thể hiện thực bằng phần cứng. Trọng tâm của chương không phải là mô tả lại FFT như một chương trình phần mềm, mà là chỉ rõ từng phép toán được ánh xạ thành khối phần cứng nào, dữ liệu đi qua đường ống theo thứ tự nào, phản hồi trễ được điều khiển ra sao, và vì sao kiến trúc R2²SDF có thể giảm số bộ nhân phức nhưng vẫn giữ được đường dữ liệu đều.

5.1 Định hướng thiết kế và tham số hệ thống

Kiến trúc được chọn là Radix-2² Single-path Delay Feedback, viết tắt là R2²SDF. Hiểu theo tiếng Việt, đây là kiến trúc một đường dữ liệu phản hồi trễ. Cách chọn này phù hợp với mục tiêu của đồ án vì nó giữ được khối butterfly Radix-2 đơn giản, nhưng lại đạt số tầng nhân phức gần với Radix-4. Ngoài ra, cơ chế phản hồi trễ giúp tổng bộ nhớ trễ chỉ còn $N - 1$ mẫu phức, thay vì phải lưu nhiều nhánh dữ liệu song song như các kiến trúc đa đường.

Trong toàn chương, các thuật ngữ và ký hiệu được dùng thống nhất như sau: N là kích thước FFT, DATA_W là độ rộng dữ liệu số cố định, D là độ sâu đường trễ của từng tầng, cnt là bộ đếm toàn cục, rot_sel là tín hiệu chọn hệ số xoay tầm thường, và CMUL là bộ nhân phức đầy đủ. Tên DATA_W được dùng cho độ rộng từ dữ liệu để tránh nhầm với ký hiệu butterfly $B_N^{k_1}$ và hệ số xoay W_N .

Bảng 5.1: Tham số chính của thiết kế FFT-256 R2²SDF

Tham số	Giá trị	Ghi chú
Kích thước FFT	$N = 256$	Tương ứng 8 tầng Radix-2.
Thuật toán	Radix-2 ² DIF FFT	Hai tầng Radix-2 được gom thành một cụm Radix-2 ² .
Kiến trúc RTL	Single-path Delay Feedback	Một mẫu phức đi vào mỗi chu kỳ xung nhịp.
Thông lượng mục tiêu	1 mẫu phức/chu kỳ xung nhịp sau khi đường ống đầy	Độ trễ ban đầu vẫn tồn tại do các đường trễ.
Thứ tự ngõ vào	Thứ tự tự nhiên	Dữ liệu vào theo thứ tự tự nhiên.
Thứ tự ngõ ra	Thứ tự đảo chữ số / đảo bit	Có thể sắp xếp lại thêm nếu hệ thống phía sau cần thứ tự tự nhiên.
Định dạng số	Q1.15, DATA_W = 16 bit	1 bit dấu và 15 bit phần phân số.
Cơ chế chia tỷ lệ	Dịch phải 1 bit sau mỗi butterfly	Giảm rủi ro tràn số; khi đánh giá biên độ cần tính cả hệ số chia tỷ lệ toàn cục.

5.2 Từ phương trình thuật toán sang khối RTL

Ở chương thuật toán, phép biến đổi Radix-2² DIF có thể được nhìn qua biến trung gian $H(k_1, k_2, n_3)$. Biến này gom hai tầng butterfly Radix-2 liên tiếp, trong đó tầng thứ hai chỉ cần nhân với các hệ số xoay tầm thường thuộc tập $\{1, -j, -1, j\}$. Đây chính là điểm giúp phần cứng tránh được một bộ nhân phức ở tầng này.

$$\begin{aligned}
 H(k_1, k_2, n_3) = & \left[x[n_3] + (-1)^{k_1} x \left[n_3 + \frac{N}{2} \right] \right] \\
 & + (-j)^{k_1 + 2k_2} \left[x \left[n_3 + \frac{N}{4} \right] + (-1)^{k_1} x \left[n_3 + \frac{3N}{4} \right] \right].
 \end{aligned} \tag{5.1}$$

Từ (5.1), có thể tách quá trình xử lý thành hai lớp phần cứng. Lớp thứ nhất là các phép cộng/trừ của butterfly Radix-2. Lớp thứ hai là phép xoay

tầm thường $(-j)^q$, với $q = k_1 + 2k_2$. Vì q chỉ nhận bốn giá trị 0, 1, 2, 3 nên phép xoay này không cần bộ nhân; chỉ cần đổi dấu và hoán đổi phần thực/phần ảo.

Sau khi tạo $H(k_1, k_2, n_3)$, biểu thức DFT có thể viết lại theo dạng

$$X[k_1 + 2k_2 + 4k_3] = \sum_{n_3=0}^{N/4-1} H(k_1, k_2, n_3) W_N^{n_3(k_1+2k_2)} W_{N/4}^{n_3 k_3}. \quad (5.2)$$

Phần $W_N^{n_3(k_1+2k_2)}$ trong (5.2) là hệ số xoay không tầm thường. Đây là nơi cần bộ nhân phức đầy đủ, vì hệ số có thể là một giá trị phức bất kỳ trên vòng tròn đơn vị. Phần $W_{N/4}^{n_3 k_3}$ thể hiện rằng sau mỗi cụm Radix-2², bài toán còn lại là một DFT ngắn hơn, tiếp tục được xử lý bởi các cụm R2²SDF phía sau.

Bảng 5.2: Ánh xạ từ thành phần thuật toán sang khối RTL

Thành phần toán học	Ý nghĩa	Khối RTL tương ứng
$\mathcal{B}_N^{k_1}(r)$	Butterfly Radix-2 tầng thứ nhất	BF2I trong tầng SDF
$(-j)^q, q = k_1 + 2k_2$	Hệ số xoay tầm thường	trivial_rotator trong BF2II
$H(k_1, k_2, n_3)$	Kết quả sau hai tầng butterfly Radix-2	Ngõ ra của một cặp BF2I + BF2II
$W_N^{n_3(k_1+2k_2)}$	Twiddle không tầm thường sau cụm Radix-2 ²	CMUL + twiddle ROM
$W_{N/4}^{n_3 k_3}$	DFT con chiều dài $N/4$	Các cụm R2 ² SDF tiếp theo
Ánh xạ chỉ số n, k	Quyết định mẫu nào được ghép butterfly với nhau	Đường trễ + cnt + tín hiệu chọn pha

5.3 Sơ đồ khối tổng thể FFT-256 R2²SDF

Với FFT-256, số tầng Radix-2 thông thường là $\log_2(256) = 8$. Trong kiến trúc R2²SDF, hai tầng Radix-2 được gom thành một cụm Radix-2², nên toàn

bộ xử lý gồm $\log_4(256) = 4$ cụm. Mỗi cụm có một BF2I, một BF2II và một bộ nhân phức đầy đủ nếu cụm đó chưa phải là cụm cuối.

Luồng xử lý tổng quát có thể viết gọn như sau:

$$\text{Ngõ vào} \rightarrow \text{BF2I}(D = 128) \rightarrow \text{BF2II}(D = 64) \rightarrow \text{CMUL}(W_{256}) \rightarrow \text{BF2I}(D = 32) \rightarrow \text{BF2II}(D = 16) \rightarrow \text{CMUL}(W_{64}) \rightarrow \text{BF2I}(D = 8) \rightarrow \text{BF2II}(D = 4) \rightarrow \text{CMUL}(W_{16}) \rightarrow \text{BF2I}(D = 2) \rightarrow \text{BF2II}(D = 1) \rightarrow \text{Ngõ ra}$$

Các đường trễ giảm theo lũy thừa của 2. Đây là đặc trưng của FFT dạng DIF: ở các tầng đầu, hai mẫu cần ghép butterfly cách nhau xa; càng về sau, khoảng cách ghép mẫu càng ngắn.

$$D = \{128, 64, 32, 16, 8, 4, 2, 1\}, \quad (5.3)$$

$$D_{\text{total}} = 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255 = N - 1, \quad (5.4)$$

$$N_{\text{CMUL}} = \log_4(N) - 1 = \log_4(256) - 1 = 3. \quad (5.5)$$

Như vậy, thay vì cần bộ nhân phức ở nhiều tầng Radix-2, thiết kế FFT-256 R²SDF chỉ cần ba CMUL đầy đủ. Cụm cuối không cần CMUL vì phần DFT còn lại có chiều dài 4, các hệ số tương ứng chỉ là các hệ số tầm thường có thể xử lý bằng BF2II.

5.4 Nguyên lý tầng SDF cơ bản

SDF là viết tắt của Single-path Delay Feedback. “Một đường dữ liệu” nghĩa là dữ liệu chỉ đi trên một luồng chính: mỗi chu kỳ xung nhịp nhận một mẫu phức, không tách thành nhiều đường song song. “Phản hồi trễ” nghĩa là một phần kết quả trung gian được đưa ngược vào đường trễ để chờ ghép với mẫu đến sau.

Một tầng SDF có độ sâu D hoạt động theo chu kỳ $2D$ chu kỳ xung nhịp. Trong nửa chu kỳ đầu, tầng chủ yếu nạp mẫu vào đường trễ. Trong nửa chu kỳ sau, mẫu mới được ghép với mẫu cũ đã được giữ lại đúng D chu kỳ xung nhịp để tạo phép butterfly. Hai kết quả của butterfly không đi ra cùng lúc như mô hình phần mềm; chúng được phân kênh theo thời gian. Một nhánh đi tiếp trong đường dữ liệu, nhánh còn lại được đưa vào phản hồi để xuất hiện ở thời điểm phù hợp.

Điểm cần nắm chắc là đường trễ không phải bộ nhớ phụ “lưu tạm cho tiện”. Nó là phần quyết định đúng cặp mẫu butterfly. Nếu D sai, hoặc tín

hiệu chọn pha nạp/tính lệch một chu kỳ xung nhịp, công thức cộng/trừ vẫn đúng trên giấy nhưng kết quả FFT sẽ sai hoàn toàn.

5.5 Thiết kế BF2I - butterfly Radix-2 cơ bản

BF2I hiện thực tầng butterfly Radix-2 đầu tiên trong mỗi cụm Radix-2². Khối này không sử dụng hệ số xoay. Nhiệm vụ của nó là cộng và trừ hai mẫu phức cách nhau D chu kỳ xung nhịp: mẫu cũ a lấy từ đường trễ và mẫu hiện tại b đi vào tầng.

Vì dữ liệu dùng định dạng Q1.15, phép cộng hai số cùng biên độ có thể làm tăng bit. Để giữ độ rộng dữ liệu cố định $\text{DATA_W} = 16$ bit, phép cộng/trừ được mở rộng dấu trước khi tính, sau đó dịch phải 1 bit để đưa kết quả về lại miền Q1.15. Với mỗi phần thực và phần ảo, khối BF2I tính:

$$y_0 = (a + b) \gg 1, \quad (5.6)$$

$$y_1 = (a - b) \gg 1. \quad (5.7)$$

Trong đó, y_0 thường là nhánh đi tiếp sang tầng sau, còn y_1 là nhánh được hồi tiếp vào đường trễ. Cách định tuyến cụ thể phụ thuộc vào pha điều khiển của SDF, nhưng tên tín hiệu nên được giữ thống nhất: a là dữ liệu từ đường trễ, b là dữ liệu ngõ vào hiện tại, y_0 là tổng đã chia tỷ lệ và y_1 là hiệu đã chia tỷ lệ.

Ở mức RTL, BF2I nên được viết như một khối tổ hợp rõ ràng cho phép cộng/trừ, kết hợp với thanh ghi đường ống ở biên tầng nếu cần đồng định thời. Không nên gán cứng lịch MUX bằng cảm tính; lịch này phải đi theo cnt và phải được kiểm tra bằng dạng sóng/testbench.

5.6 Thiết kế BF2II và bộ xoay tâm thường

BF2II là khối tạo ra lợi thế chính của Radix-2². Nó vẫn thực hiện một butterfly Radix-2, nhưng ở nhánh cần xoay pha, hệ số chỉ thuộc tập tâm thường $\{1, -j, -1, j\}$. Vì vậy, BF2II không cần CMUL. Phép nhân với các hệ số này được hiện thực bằng đổi dấu bù 2 và hoán đổi phần thực/phần ảo.

Tương tự BF2I, dữ liệu sau cộng/trừ được chia tỷ lệ để giảm rủi ro tràn

số:

$$u_0 = (a + b) \gg 1, \quad (5.8)$$

$$u_1 = (a - b) \gg 1. \quad (5.9)$$

Do SDF là kiến trúc phân kênh theo thời gian, ngõ ra của BF2II không nên hiểu như một phương trình tĩnh duy nhất. Ở pha tính, u_0 đi tiếp trong đường dữ liệu, còn u_1 được ghi vào đường trễ. Ở pha còn lại, dữ liệu u_1 đã trễ đủ D chu kỳ xung nhịp được đọc ra và đi qua `trivial_rotator` trước khi đi tiếp. Nói ngắn gọn: BF2II vừa tính butterfly, vừa sắp lịch xuất hai nhánh kết quả theo thời gian.

Tín hiệu chọn xoay được ký hiệu là `rot_sel` = $q[1 : 0]$, với $q = k_1 + 2k_2$. Bảng 5.3 trình bày ánh xạ phần cứng của `trivial_rotator`.

Bảng 5.3: Ánh xạ hệ số xoay tầm thường trong BF2II

<code>rot_sel</code> / q	$C_q = (-j)^q$	Ánh xạ phần thực	Ánh xạ phần ảo
0	1	$out_re = in_re$	$out_im = in_im$
1	$-j$	$out_re = in_im$	$out_im = -in_re$
2	-1	$out_re = -in_re$	$out_im = -in_im$
3	j	$out_re = -in_im$	$out_im = in_re$

Bảng trên cũng cho thấy vì sao phép xoay này rẻ về phần cứng: không có phép nhân nào xuất hiện. Khối chỉ cần MUX, dây nối và logic đảo dấu.

5.7 Thiết kế bộ nhân phức CMUL

Sau mỗi cụm BF2I + BF2II, ngoại trừ cụm cuối, dữ liệu phải nhân với hệ số xoay không tầm thường. Khối này được gọi là CMUL. Để tránh nhầm ký hiệu, chương này dùng $z = z_{re} + jz_{im}$ cho dữ liệu vào và $t = t_{re} + jt_{im}$ cho hệ số xoay đọc từ ROM.

$$z \cdot t = (z_{re} + jz_{im})(t_{re} + jt_{im}), \quad (5.10)$$

$$y_{re} = z_{re}t_{re} - z_{im}t_{im}, \quad (5.11)$$

$$y_{im} = z_{re}t_{im} + z_{im}t_{re}. \quad (5.12)$$

Cách triển khai trực tiếp cần bốn bộ nhân thực và hai bộ cộng/trừ thực. Trên FPGA, bốn bộ nhân này thường ánh xạ vào DSP slice. Trên ASIC, đây là một trong các khối ảnh hưởng mạnh đến diện tích phần cứng, công suất và định thời. Vì vậy, việc giảm số CMUL xuống còn $\log_4(N) - 1$ là lợi ích quan trọng nhất của kiến trúc R2²SDF.

Với số cố định Q1.15, tích của hai số 16 bit tạo ra kết quả trung gian rộng hơn 16 bit. Do đó, CMUL cần có bước cắt bit hoặc làm tròn về Q1.15. Nếu chỉ cắt bớt bit, phần cứng đơn giản hơn nhưng nhiều lượng tử lớn hơn. Nếu làm tròn, kết quả đẹp hơn nhưng cần thêm logic. Ở bản RTL cơ sở, có thể dùng cắt bớt bit trước để dễ kiểm chứng, sau đó đánh giá lại sai số bằng testbench.

5.8 Twiddle ROM và sinh địa chỉ

Twiddle ROM lưu các hệ số W_M^r ở dạng số cố định Q1.15. Vì đường ống xử lý một mẫu mỗi chu kỳ xung nhịp, cách dễ triển khai và dễ đồng định thời nhất là dùng một ROM riêng cho từng tầng CMUL. Với FFT-256, có ba CMUL nên cần ba nhóm hệ số chính: W_{256} , W_{64} và W_{16} .

$$T_M(q, r) = W_M^{qr}, \quad q \in \{0, 1, 2, 3\}. \quad (5.13)$$

Bảng 5.4: Nhóm twiddle ROM theo từng tầng CMUL

Tầng CMUL	Sau cụm	Chiều dài DFT hiện tại M	Hệ số cần lưu	Ghi chú
0	Cụm 0	256	W_{256}^{qr}	$q = 0$ có thể bỏ qua vì hệ số bằng 1.
1	Cụm 1	64	W_{64}^{qr}	Địa chỉ ROM ngắn hơn tầng 0.
2	Cụm 2	16	W_{16}^{qr}	Nhiều hệ số rơi vào trường hợp tầm thường.
-	Cụm 3	4	Không cần ROM	Cụm cuối chỉ còn hệ số tầm thường.

Trong RTL, địa chỉ ROM không nên sinh bằng phép chia hoặc modulo phức tạp vì các phép này không thân thiện với định thời. Cách hợp lý hơn là dùng cnt đồng bộ toàn cục và lấy các bit tương ứng cho từng tầng. Hai bit cao của từng cụm xác định q hoặc `rot_sel`, còn các bit thấp hơn tạo r , tức địa chỉ đọc ROM.

Một điểm cần thống nhất khi viết mã là tên tín hiệu ROM. Ví dụ, tầng 0 dùng `tw_re_s0` và `tw_im_s0`, tầng 1 dùng `tw_re_s1` và `tw_im_s1`, tầng 2 dùng `tw_re_s2` và `tw_im_s2`. Không nên trộn các tên như `W_ROM`, `twiddle_mem`, `coef_rom` trong cùng một chương hoặc cùng một mã RTL nếu chúng chỉ cùng một ý nghĩa.

5.9 Đường trễ và bộ nhớ phản hồi

Trong $R2^2SDF$, đường trễ là phần giữ mẫu cũ để ghép với mẫu mới ở đúng khoảng cách D . Mỗi tầng SDF cần một đường trễ có độ sâu D mẫu phức. Với FFT-256, các giá trị D lần lượt là 128, 64, 32, 16, 8, 4, 2 và 1. Tổng số ô nhớ phức cần dùng là:

$$D_{\text{total}} = \sum_i D_i = 2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = 255 = N - 1. \quad (5.14)$$

Trong phiên bản RTL cơ sở của đồ án, đường trễ nên được hiện thực bằng thanh ghi dịch. Cách này tốn nhiều thanh ghi hơn so với RAM, nhưng

rất dễ quan sát trên dạng sóng: dữ liệu đi qua từng chu kỳ xung nhịp và xuất hiện ở ngõ ra sau đúng D chu kỳ. Với giai đoạn cần kiểm chứng thuật toán, sự rõ ràng này đáng giá hơn việc tối ưu tài nguyên quá sớm.

Sau khi thiết kế đã đúng chức năng, các đường trễ dài như $D = 128, 64, 32$ và 16 có thể chuyển sang RAM vòng hoặc BRAM để giảm tài nguyên logic. Các đường trễ ngắn như $D = 8, 4, 2$ và 1 vẫn có thể giữ dạng thanh ghi dịch vì chi phí nhỏ và định thời thường tốt hơn.

Nói thẳng: nếu chuyển sang RAM quá sớm, phần gỡ lỗi sẽ khó hơn rất nhiều. Với đồ án này, ưu tiên đúng chức năng trước, tối ưu tài nguyên sau. Đây là cách làm an toàn và thực tế hơn.

5.10 Bộ điều khiển

Một ưu điểm của R2²SDF là bộ điều khiển không cần FSM lớn. Do lịch xử lý có tính tuần hoàn, một bộ đếm toàn cục cnt có độ rộng $\log_2(N)$ bit có thể dùng đồng thời cho ba việc: chọn pha nạp/tính của từng tầng trễ, tạo rot_sel cho BF2II và sinh địa chỉ đọc twiddle ROM.

Với FFT-256, cnt có 8 bit, ký hiệu cnt[7:0]. Mỗi tầng trễ lấy một bit của cnt để xác định pha hiện tại. Các cặp bit lân cận được dùng làm rot_sel cho BF2II hoặc làm một phần địa chỉ ROM. Bảng 5.5 trình bày cách dùng bit điều khiển theo từng tầng.

Bảng 5.5: Gợi ý ánh xạ bit điều khiển từ bộ đếm toàn cục cnt

Tầng delay D	Bit cnt dùng cho pha	Hai bit điều khiển rot_sel	Khối sử dụng
128	cnt[7]	cnt[7:6]	BF2I cụm 0
64	cnt[6]	cnt[7:6]	BF2II cụm 0
32	cnt[5]	cnt[5:4]	BF2I cụm 1
16	cnt[4]	cnt[5:4]	BF2II cụm 1
8	cnt[3]	cnt[3:2]	BF2I cụm 2
4	cnt[2]	cnt[3:2]	BF2II cụm 2
2	cnt[1]	cnt[1:0]	BF2I cụm 3
1	cnt[0]	cnt[1:0]	BF2II cụm 3

Bảng này nên được xem là lịch điều khiển logic ở mức kiến trúc. Khi

viết RTL, cần kiểm tra lại quy ước đếm bắt đầu từ 0 hay từ 1, cạnh reset, và thời điểm `valid_in` bắt đầu. Chỉ cần lệch một chu kỳ xung nhịp giữa `cnt` và dữ liệu, toàn bộ lịch phản hồi trở sẽ sai.

Vì vậy, bộ điều khiển nên đi kèm tín hiệu `valid_pipe` để đánh dấu dữ liệu hợp lệ sau khi đường ống đầy. Ngõ ra không nên được xem là hợp lệ ngay từ chu kỳ xung nhịp đầu tiên, vì các đường trễ cần thời gian nạp trạng thái ban đầu.

5.11 Ước lượng tài nguyên

Bảng 5.6 tóm tắt tài nguyên chính của thiết kế FFT-256 R2²SDF. Đây là ước lượng ở mức kiến trúc, dùng để định hướng thiết kế RTL trước khi tổng hợp. Con số thực tế sau tổng hợp sẽ phụ thuộc vào cách viết RTL, cách ánh xạ đường trễ, cách làm tròn số cố định và công nghệ mục tiêu.

Bảng 5.6: Ước lượng tài nguyên cho FFT-256 R2²SDF

Thành phần	Số lượng cho FFT-256	Nhận xét
BF2I	4	Một khối cho mỗi cụm Radix-2 ² .
BF2II	4	Tích hợp <code>trivial_rotator</code> .
CMUL	3	Tương ứng $\log_4(256) - 1$.
Thanh ghi trễ phức	255 mẫu phức	Tương ứng tổng $D = N - 1$ nếu dùng thanh ghi dịch.
Twiddle ROM	3	Lần lượt cho W_{256} , W_{64} và W_{16} .
Bộ đếm toàn cục	8 bit	Vì $\log_2(256) = 8$.

Nếu mỗi mẫu phức gồm hai phần Q1.15, một mẫu phức cần 32 bit. Vì vậy, riêng đường trễ dạng thanh ghi dịch cần khoảng

$$255 \times 32 = 8160 \text{ bit} \quad (5.15)$$

thanh ghi. Con số này chưa lớn ở mức mô phỏng và kiểm chứng, nhưng trên FPGA/ASIC vẫn nên tối ưu ở các đường trễ dài nếu thiết kế cần tiết kiệm tài nguyên.

5.12 Tổng kết chương

Chương này đã chuyển phần thuật toán Radix-2² DIF FFT sang kiến trúc RTL cụ thể cho FFT-256. Các khối chính của đường dữ liệu gồm BF2I, BF2II, `trivial_rotator`, CMUL, twiddle ROM, đường trễ và bộ điều khiển cnt. Điểm cốt lõi của thiết kế là giữ luồng dữ liệu một mẫu phức mỗi chu kỳ xung nhịp, dùng phản hồi trễ để ghép đúng cặp butterfly, và chỉ dùng CMUL ở các vị trí thật sự cần hệ số xoay không tầm thường.

Về hướng triển khai tiếp theo, RTL nên được xây từ dưới lên: kiểm chứng BF2I và BF2II trước, sau đó kiểm chứng `trivial_rotator`, CMUL, twiddle ROM, từng tầng SDF, rồi mới ghép toàn bộ FFT-256. Nếu nhảy thẳng vào module mức đỉnh mà chưa kiểm chứng từng khối, lỗi định thời và lỗi lịch điều khiển sẽ rất khó khoanh vùng.